This lab may seem a little difficult[T1], so please start early. Once you get the hang of context switching, you'll be in a good position to understand how other languages use user-space threads to simplify concurrent programming. C++ does not have a user-space thread implementation in the standard library.

See for example, *goroutines* in GoLang[T2] and *green threads* in Java 1.0 if you are interested. There are lots of subtle issues in implementing user-space threads. For one, because user-space threads are *invisible* to the OS kernel[T3], if any user-space thread makes a system call, it can potentially block the entire process. This can drastically affect program latency. Also, if user-space threads are purely *cooperative*, then it's possible for one thread to never yield the CPU and starve all other threads. Real implementations such as *goroutines* handle these issues.

---

## 1 Introduction

The objective of this lab is to implement a simple **user-space**, **cooperative**, **multi-tasking** thread library. Let's take a look at what each of these words mean one by one.

- **User-space** means that this library will be linked against user programs—such as the programs you write, editors, web browsers, etc., and the code in the library will be executed as part of your program. It means that we *won't* need any modifications in the kernel.
- **Cooperative** means that the workers—i.e., the independent units of execution in your program, cooperatively yield control to other workers. This is different from how operating systems are *usually* implemented. Typically, the OS preempts a worker thereby taking control from it, rather than relying on the worker to willingly yield the control. The reason that OSes typically do this (known as *preemptive multitasking*) is to be fair in CPU allocation to all the workers—both within a process and globally. The workers may be faulty or malicious and never yield the CPU to another worker.

*Preemptive multitasking*, however, comes at a cost of higher complexity because the OS is typically unaware of the optimal time-slice for a worker. To avoid complexity in implementation, we've chosen a cooperative design in which workers explicitly yield control. You can read more about these two design choices on **StackOverflow**.

- **Multi-tasking** is a catch-all term for multi-threading and multi-processing. Strictly speaking, in this lab we won't create *threads* that are independently schedulable entities; rather what we'll create are *tasks*. A task is defined as a unit of work which the parallel runtime (the OS, or a parallel runtime user-space library like the one in this assignment) guarantees will get completed. A parallel runtime is defined as the set of function calls and their behavior which allows program code to be parallelized.

In some user-space parallel runtimes, *tasks* share the same stack. However, in this library which you'll implement, each worker *explicitly* allocates the stack for its use.

Because machine registers are shared between the various tasks (because the library executes on the same processor), we need to save the state of the machine, also called the *context*, whenever we switch to a different task. These contexts, which include the values of various architectural registers, are stored in a specific data structure in the library. Therefore, whenever we want to switch the control over to a new task we need to perform two steps: (1) save the current context, and (2) switch over to the new context stored in the internal data structure. Moreover, since the context holds the complete information of the state of the machine when the context was captured, when we resume from a context it appears as if we went back in time and started from where we had left off, i.e. from the point of context capture.

In this lab, you are **not** expected to write various functions which deal with saving and restoring the snapshots, rather you only need to deal with resuming the correct context at the correct time. We'll use the **ucontext.h** header provided by the standard C library, i.e. **libc**, which is available on

**UNIX** systems for this lab. More details on the header can be found in the **man** pages. Please refer to the **man** page for **makecontext** for an example program. You can use whatever **libc** calls and **syscalls** that you want for this lab. However, please note that this lab can be completed using only three function calls declared in **ucontext.h**: **swapcontext**, **getcontext**, and **makecontext**. The TAs will cover the semantic of all these three functions in the lab sessions to get you started.

Since many of you might not have used these functions before, and many of you might not have written a parallel runtime before, we recommend you to **get started as early as possible** since this lab will take some time.

---

## 2 The Library Calls

The **ucontext.h** headers and its functions operate on objects of the type **ucontext_t**. More details of this type can be found in the **man** pages for **getcontext**. This structure represents a captured context of the machine.

- **getcontext(ucontext_t *ucp)**: This function takes a pointer to an object of the type **ucontext_t** and populates the various field of the object such that after the call to **getcontext**, it contains a valid captured context of the machine. Note that this context can then be used to restore the machine state to the point where the context was captured, i.e. to the point just *after* the call to **getcontext**.
  You can think of this "state" as comprising of the processor state (registers) + some process-specific state such as the process's signal mask. When the saved state is activated, it appears as if **getcontext()** returned again.
  Note that there's only one process that is multiplexing different threads of control, so all the "threads" inherit the same signal mask.
- **makecontext(ucontext_t *, (void (*)()) func, <nargs>, arg1,...)**: This function takes a pointer to an object of the type **ucontext_t** and modifies it. The object which is passed to this function must be a valid context.

This function also takes in a function pointer, the number of arguments to be passed to the function, and all the arguments which are to be passed to the function. This function modifies the context which was passed to it (via the pointer) such that whenever this context is resumed, the control jumps to the first instruction of the function whose function pointer was passed. The function which was passed gets called with the arguments specified in the call to **makecontext**.
However, note that although this call modifies the context such that it executes the right function when it is resumed, it does **not** set up the stack for the call to this function. This task is left to the users of the **makecontext**, which happens to be you in this case.

- To setup the stack you need to set the pointer **context.uc_stack.ss_sp** to some space in memory which can be used for the stack (you can simply **malloc** some space for this).
- You need to set **context.uc_stack.ss_size** to the size of the stack space which you've allocated.
- You need to set **context.uc_stack.ss_flags** to 0 because the child task is not required to handle any signals.
- Finally, you need to set **context.uc_link** to **NULL** to indicate that once the function finishes execution, the worker can simply terminate. You can optionally make this field point to another **ucontext_t** object which would be resumed after the specified function exits. Setting this field to **NULL** ensures that the worker would exit after executing the specified function.

Once you've used the **makecontext** call and completed the four tasks listed above, the context which you modified is finally ready to be used.

```
getcontext(ucp);
 ucp->uc_stack.ss_sp = (char *) malloc(STACK_SZ);
 ucp->uc_stack.ss_size = STACK_SZ;
 ucp->uc_stack.ss_flags = 0;
 ucp->uc_link = NULL;
 makecontext(ucp, (void (*)()) fn, 2, arg1, arg2);
```

- **swapcontext(ucontext_t *from, ucontext_t *to)**: This function takes two pointers to two distinct contexts as its arguments. It then saves the current context, i.e. the context of the caller, in the first argument and restores the execution from the second context. Note that when the execution is restored, the control would jump to wherever the second context was created. This means that if the call to **swapcontext** doesn't fail, then the control is not guaranteed to return to the call site.

---

## 3 API Details

The API of the library that you need to implement is listed in the **README.md** file in the starter code. The function signatures of various calls which you need to provide are mentioned in **threading.h**. The TAs will also go over the function signatures in the labs.

- **void t_init()**: This is the first function which is called by any program that uses the library. You can use this function to initialize various data structures which your library may or may not be using.
- **int32_t t_create(fptr foo, int32_t arg1, int32_t arg2)**: This call is used to create a task represented by the function **foo** with two arguments, **arg1** and **arg2**. Note that the **fptr** type is a function pointer of the type
- **void (*fptr)(int32_t, int32_t)**
- i.e. foo *must* be a pointer to a function which returns nothing, i.e. void, and which *must* take two 32-bit signed integer arguments **arg1** and **arg2**. For an example of such a function and the usage of **t_create**, see **main.c**.
- **t_yield()**: This function is called by a worker to indicate that it can relinquish the control over to other workers at the call site of **t_yield**. The existence of this function tells us that this library is a *cooperative* multi-tasking library.

- **t_finish()**: This function is called by all workers, except the main worker which spawns all the tasks, at the very end to indicate the completion of the assigned task.

Please refer to **threading.h** for more details about the exact API that you need to implement.

threading.h

```
#include <stdint.h>
#include <stdlib.h>
#include <string.h>
#include <ucontext.h>

#ifndef COOPERATIVE_MULTITASKING
#define COOPERATIVE_MULTITASKING

#define STK_SZ  4096
#define NUM_CTX 16

enum context_state {
  INVALID = 0,   // The context entry is invalid
  VALID = 1,     // The context entry is valid and ready to be used
  DONE = 2,      // This context has completed its work
};

struct worker_context {
  enum context_state state;
  ucontext_t context;
};

extern struct worker_context contexts[NUM_CTX];
extern uint8_t current_context_idx;

typedef void (*fptr) (int32_t, int32_t);
typedef void (*ctx_ptr) (void);

void t_init ();

int32_t t_create (fptr foo, int32_t arg1, int32_t arg2);

int32_t t_yield ();
```

```
void t_finish ();
#endif
```

---

## 4 The Starter Code

The stater code comes with seven files as described below.

- **Problem.pdf**: This document which describes the task for Lab 3.
- **main.c**: The code which uses the library that you'll implement. You are **not** allowed to modify this file as this is the test application which we'll use to evaluate your submission. However, if you want to write other programs which use your implementation for testing purposes, you can always copy this file somewhere else and then make changes to it.
- **Makefile**: The Makefile to build the library and the executable. You can modify this file if you so desire, but you probably won't have to.
- **README.md**: This file provides more details about the task.
- **threading.c**: This file holds the implementation of the library. You are expected to modify this file however you want. To get you started, we've provided sections in this file where you can write your code. These sections are marked with **TODO**s.
- **threading_data.c**: This file defines all the variables which your library implementation may or may not use. You can modify this file if you want, but you probably won't have to. However, note that if you add/remove various variables to/from this file, make sure to add the corresponding **extern** declarations for them in **threading.h**.
- **threading.h**: This is the header file for the library which is used by **main.c**. This file has all the information about the various calls that you've to implement. You can **only add** declarations in this file but you are not allowed to remove any of the function declarations. If you do remove any of these declarations, then the code would fail to compile.

The starter code is structured as a **dynamically linked** (it's linked at runtime) **shared** library. The code that you'll write in **threading.c** will be used to compile a shared library named **libthreading.so**. This library can then we used with any other program.

**main.c** implements one such program that uses the library. To use the functions supported by the library, **main.c** includes **threading.h**. When **main.c** is compiled to generate the executable **main**, it is linked with the shared library.

As a result, whenever **main** calls one of the library functions, the function is invoked in the **libthreading.so** shared library. The **Makefile** handles all of the compilation logic. The output of the **Makefile** is one executable, **main**, and one dynamically linked shared library, **libthreading.so**.

---

# 5 Hints

## 5.1 An illustrative example

Consider the example code shown on the right. When this code is compiled and run with an argument, here's what happens when argc > 1.

---

main prints main: swapcontext(&uctx_main, &uctx_func2) and swaps to uctx_func2. (This

starts func2. func2 runs.)

prints func2: started

prints func2: swapcontext(&uctx_func2, &uctx_func1)

swapcontext(&uctx_func2, &uctx_func1) transfers to func1.

func1 runs

prints func1: started

prints func1: swapcontext(&uctx_func1, &uctx_func2)

swapcontext(&uctx_func1, &uctx_func2) transfers back to func2.

Back in func2 after its swapcontext returns

prints func2: returning

func2 returns. (Because uc_link is NULL (argc > 1), there is no successor context.

Returning from a ucontext with uc_link == NULL terminates the thread/process via

setcontext semantics; the program ends here. Control does not go to main, and

func1's "returning" line is never printed.)

---

When the program is run without arguments, its output is the following.

main: swapcontext(&uctx_main, &uctx_func2)

func2: started

func2: swapcontext(&uctx_func2, &uctx_func1)

func1: started

func1: swapcontext(&uctx_func1, &uctx_func2)

func2: returning

func1: returning

main: exiting

---

Key points:

- With no argument, returning from func2 goes to func1 (via uc_link).
- Returning from func1 goes to main (via uc_link).
- main then continues and exits normally.

---

An example of the ucontext API

```
#include <stdio.h>

#include <stdlib.h>

#include <ucontext.h>


static ucontext_t uctx_main, uctx_func1, uctx_func2;


static void func1 (void) {

  printf ("%s: started\n", __func__);

  printf ("%s: swapcontext(&uctx_func1, &uctx_func2)\n", __func__);

  swapcontext (&uctx_func1, &uctx_func2);

  printf ("%s: returning\n", __func__);

}
```

```
static void func2 (void) {
  printf ("%s: started\n", __func__);
  printf ("%s: swapcontext(&uctx_func2, &uctx_func1)\n", __func__);
  swapcontext (&uctx_func2, &uctx_func1);
  printf ("%s: returning\n", __func__);
}

int main (int argc, char *argv[]) {
  char func1_stack[16384];
  char func2_stack[16384];

  getcontext (&uctx_func1);
  uctx_func1.uc_stack.ss_sp = func1_stack;
  uctx_func1.uc_stack.ss_size = sizeof (func1_stack);
  uctx_func1.uc_link = &uctx_main;
  makecontext (&uctx_func1, func1, 0);

  getcontext (&uctx_func2);
  uctx_func2.uc_stack.ss_sp = func2_stack;
  uctx_func2.uc_stack.ss_size = sizeof (func2_stack);
  /* Successor context is f1(), unless argc > 1 */
```

```
  uctx_func2.uc_link = (argc > 1) ? NULL : &uctx_func1;
  makecontext (&uctx_func2, func2, 0);

  printf ("%s: swapcontext(&uctx_main, &uctx_func2)\n", __func__);
  swapcontext (&uctx_main, &uctx_func2);

  printf ("%s: exiting\n", __func__);
  exit (EXIT_SUCCESS);
}
```

---

### 5.2 Experiment with *libthreading.a*

Download **libthreading.a** and compile it with **main.c** as follows:

```
gcc main.c libthreading.a -o main
./main
```

You can change the program in *main.c* to experiment with our threading library. The library is **not** vey robust so don't try anything fancy. But you should get a good feel for context switching. Send me your questions and I'll answer them during lecture.

---

### 5.3 Code organization

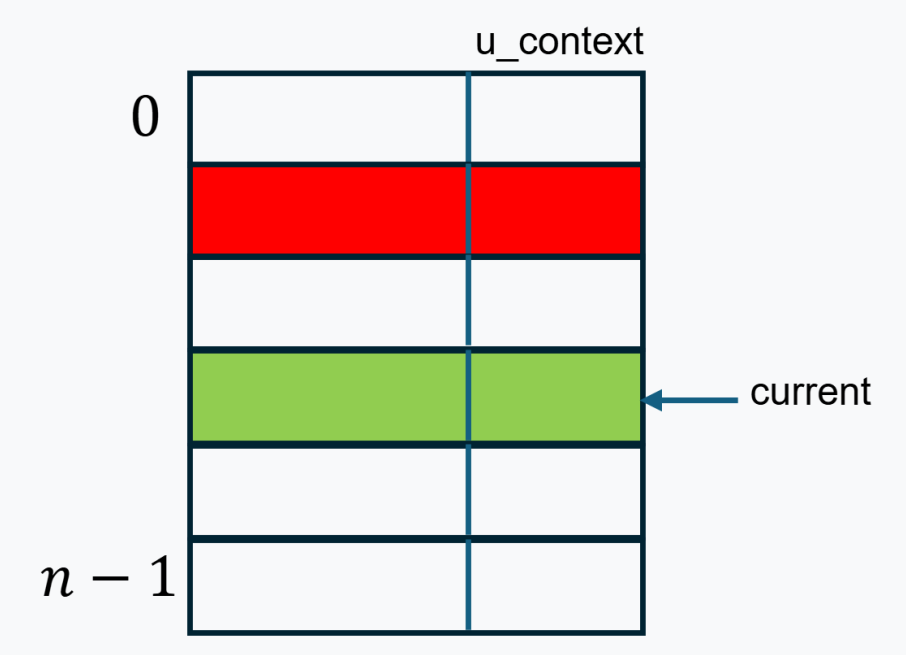
Please note that the hints provided here are merely *suggestions*. You are under **no** obligation to follow them. You can use the **contexts** array defined in **threading_data.c** to store all the contexts. You can also use

**current_context_idx** to keep track of which context is active right now. You may assume that:

- All worker threads are started before the main thread calls **t_yield()**.
- The context of the main thread is in a fixed index in the proc table, say 0. This naturally results in the outcome that when all worker threads finish, control returns to the main thread.

Here's a diagram of the *proctable* that contains an entry for each worker thread. Slots in the table correspond to worker threads and a slot contains a **ucontext_t** that saves the context of the thread.

proctable:




---

- **t_init()**: You can use this function to initialize various data structures that your library uses such as **contexts** and **current_context_idx**.
- **t_create()**: In this function you can first find an empty entry in the **contexts** array. You can use the **state** field of the **struct worker_context** and **enum context_state** to track which

entries are unused, i.e. **INVALID**. Then you can initialize this entry using **getcontext**. After initializing the entry you can then use **makecontext** to modify the context such that it calls the function passed as the argument to **t_create()**. Make sure to also allocate the stack for the context here. You may also *optionally* choose to schedule the new context at this point itself.

- **t_yield()**: In this function you can first update the current context (whose index is stored in **current_context_idx**) by taking a fresh snapshot using **getcontext**. Then you can search for a **VALID** context entry in the **contexts** array and use **swapcontext** to switch to it. After this you can then compute the number of contexts in the **VALID** state and return the count. Note that since the switch happens before you compute the number of contexts in the **VALID** state, when the original context is resumed, the calculation of this count will be done at that point.
- **t_finish()**: In this function you can first **free** the stack allocated for the current context. Then you can reset the context entry to all zeros by using **memset**.

---

t_create:

```
int32_t t_create(fptr foo, int32_t arg1, int32_t arg2) {
 // the caller uses the current slot in the proc table
 next = find_next_free();
 if(next == curr) return ERROR; // no free slots
 use makecontext to create a frame for foo(arg1, arg2)
 return SUCCESS;
}
```

---

t_yield:

```
int t_yield() {
  // the caller uses the current slot in the proc table
  next = find_next_valid();
  if(next == curr) return ERROR; // only caller remains
  atomically:
    save caller's context in current;
    restore context from next;
  return SUCCESS;
}
```

---

t_finish:

```
void t_finish() {
  free the entry in slot current;
  // there's at least one for the main thread
  next = find_next_valid();
  use setcontext to set context with proctable[next]
}
```

---

## 6 Rubric

The assignment is worth 100 points out of which 20 points are for code which correctly compiles, while 70 points are for the correct implementation. The final 10 points are for code which is free of any memory leaks.

Note that since the Address Sanitizer is not fully complaint with the **ucontext.h** library, we'll use another tool called **valgrind** to check for leaks. To use this tool, you need to install it on your machine by running **sudo apt-get install valgrind**. However, note that **valgrind** **does not play nice with ucontext.h** by default. Therefore, we'll only use it to detect memory leaks and we'll ignore all the other errors that it reports. To run **valgrind** you can execute

```
valgrind --leak-check=full --show-leak-kinds=all
--track-origins=yes ./main
```

after building the code. If your code is free of any memory leaks, you should see

```
All heap blocks were freed -- no leaks are possible
```

printed at the end. If after running **valgrind** you can see this message, then you'll get the 10 points for leak free code. If you face any weird behavior from **valgrind**, please mention it on the Discord chat. It's possible that **valgrind** cannot be used, at least not without some **valgrind** specific code changes, for this lab. If this happens to be the case, we'll remove this component and the 10 points for this components would get merged with the 70 points for the code, bringing the total to 80 points.

---

## What to Submit:

Please start this lab assignment early and ask your TAs for help. It is quite likely that you'll take some time to complete this task. Good luck and Gig 'em! You are required to submit the following four files for Lab 3.

- **threading.c**

- **threading.h**
- **threading_data.c**
- **Makefile**

---